Library Information Management System
LIMS

Testing Plan & Log

March 11, 2026

Test Plan Overview

Introduction:

This document identifies 14 test cases designed to challenge the internal functionality and user interface of LIMS Iteration One. The project team will engage a tester to conduct these tests to verify the overall operation of the Catalog and Loan Management modules. The tester will utilize this document to execute the tests as outlined; upon completion, the team will evaluate the results to produce a Test Summary Report and a Testing Incident Report for any failed cases.

Features not to be tested:

The following features are out of scope for the current iteration and will not be tested:

- Advanced loan features such as reservations and renewals.
- Automated email or SMS notifications for overdue items.
- Full client-facing self-service portal (public search and online account management).
- Integration with external third-party catalog services.
- Advanced analytics dashboards.

Environmental needs:

To successfully execute these tests, the following environment is required:

- Database Management: XAMPP stack providing MariaDB/MySQLi.

- Web Server: Apache (via XAMPP).
- Development Tools: PHP 7.4 or higher and a PHP IDE/editor (e.g., PhpStorm).
- Browser: Modern web browser (Google Chrome, Mozilla Firefox, or Microsoft Edge).

Testing tasks & schedule:

Farhan will complete the test cases for the LIMS system on March 11, 2026. Testing will take place in the local development environment using the LIMS application currently installed and configured on the project workstation.

Test Case #: 1				
Test Description: Verify that a staff member can log in successfully with valid credentials to access protected system areas.				
Use Case: Staff Login				
Functional Requirements Tested: Security and Access Control - Provide multi-level security for Administrators and Staff.				
Acceptance Criteria: The user is redirected to the home page, and their username is displayed in the navigation bar.				
Special Requirements: Requires an existing user record in the `users` table.				
Test Result: Pass				
Issues/Comments: N/A				
Tested By: Farhan		**Date Tested:** March 11, 2026		
Test Case Details				
Screen/Field/Form	**Action**	**Data Value**	**Expected Result**	**Actual Result**
index.php?page=login	Enter valid username and password, click "Sign In"	**Username:** farhan **Password:** 123	Successful login; redirect to `index.php` (Home).	Worked as expected.

Test Case #: 2				
Test Description: Verify that the system blocks login attempts using incorrect passwords and provides appropriate feedback.				
Use Case: Staff Login				
Functional Requirements Tested: Security and Access Control - Role-based authentication via PHP sessions.				
Acceptance Criteria: An error message "Invalid username or password" is displayed on the login screen.				
Special Requirements: N/A				
Test Result: Pass				
Issues/Comments: N/A				
Tested By:Farhan		**Date Tested:** March 11, 2026		
Test Case Details				
Screen/Field/Form	**Action**	**Data Value**	**Expected Result**	**Actual Result**
index.php?page=login	Enter valid username with an incorrect password.	**Username:** farhan **Password:** wrongpass	System displays: "Invalid username or password".	Worked as expected.

Test Case #: 3				
Test Description: Verify that the "Add Staff" functionality is restricted only to users with the 'ADMIN' role.				
Use Case: Register New Staff				
Functional Requirements Tested: Security and Access Control - Multi-level security for Administrators and Staff.				
Acceptance Criteria: If a user is not an Admin, the system should block the route and display "Access Denied".				
Special Requirements: Requires a user logged in with a 'USER' role (non-admin).				
Test Result: Pass				
Issues/Comments: N/A				
Tested By: Fardin Sahriar Al Rafat		**Date Tested:** March 11, 2026		
Test Case Details				
Screen/Field/Form	**Action**	**Data Value**	**Expected Result**	**Actual Result**
index.php?page=register	Log in as a 'USER' role and attempt to navigate to the Register page.	**Role:** USER	System displays: "Access Denied: Only Administrators can create new accounts.	Worked as expected.

Test Case #: 4				
Test Description: Verify that a staff member can successfully add a new book record to the system catalog.				
Use Case: Add Book				
Functional Requirements Tested: Catalog Management – Add book records including title, type, and publisher.				
Acceptance Criteria: The book is saved to the `books` table and appears at the top of the Book Inventory list.				
Special Requirements: User must be logged in as Staff or Admin.				
Test Result: Pass				
Issues/Comments: N/A				
Tested By: Farhan			**Date Tested:** March 11, 2026	
Test Case Details				
Screen/Field/Form	**Action**	**Data Value**	**Expected Result**	**Actual Result**
index.php?page=books_create	Enter required book details and click "Save Book".	**Title:** Clean Code **Type:** Programming **Publisher:** Prentice Hall	Redirect to `index.php?page=books`; new book is visible in the table.	Worked as expected.

Test Case #: 5				
Test Description: Verify that the system prevents saving a book record when required fields (Title, Type, or Publisher) are left empty.				
Use Case: Add Book				
Functional Requirements Tested: Input validation and data integrity for catalog entries.				
Acceptance Criteria: The system blocks submission and displays an error message: "All fields except supplier are required."				
Special Requirements: N/A				
Test Result: Pass				
Issues/Comments: N/A				
Tested By: Farhan			**Date Tested:** March 11, 2026	
Test Case Details				
Screen/Field/Form	**Action**	**Data Value**	**Expected Result**	**Actual Result**
index.php?page=books_create	Leave the "Title" field empty and click "Save Book".	**Title:** [Empty] **Type:** Programming **Publisher:** Pearson	Form does not submit; error message "All fields except	Worked as expected.

			supplier are required" appears.	

Test Case #: 6				
Test Description: Verify that a staff member can update existing book details (e.g., correcting a publisher's name).				
Use Case: Update Book				
Functional Requirements Tested: Catalog Management – Update book records.				
Acceptance Criteria: The updated information persists in the database and is reflected in the UI book inventory list.				
Special Requirements: Requires at least one existing book in the database.				
Test Result: Pass				
Issues/Comments: N/A				
Tested By: Farhan			**Date Tested:** March 11, 2026	
Test Case Details				
Screen/Field/Form	**Action**	**Data Value**	**Expected Result**	**Actual Result**
index.php?page=books_edit&id=1	Change the "Publisher" name and click "Update Book".	**New Publisher:** Pearson Education	Redirect to inventory list; ID #1 shows "Pearson Education" as the publisher.	Worked as expected.

Test Case #: 7				
Test Description: Verify the "Smart Deletion" logic for Suppliers: the system must block the deletion of a supplier if they are currently linked to books in the catalog.				
Use Case: Delete Supplier				
Functional Requirements Tested: Smart deletion and referential integrity.				
Acceptance Criteria: The system catches the exception, blocks the deletion, and redirects the user with an error “currently in use” parameter.				
Special Requirements: Requires a supplier record that is assigned to at least one book.				
Test Result: Pass				
Issues/Comments: N/A				
Tested By: Farhan			**Date Tested:** March 11, 2026	
Test Case Details				
Screen/Field/Form	**Action**	**Data Value**	**Expected Result**	**Actual Result**

index.php?page=suppliers	Click "Delete" on a supplier linked to books.	**Supplier ID: 1**	Redirect to supplier list with error	Worked as expected.

Test Case #: 8				
Test Description: Verify that the system prevents the deletion of a book that is currently linked to existing loan history records to maintain data integrity.				
Use Case: Delete Book				
Functional Requirements Tested: Data integrity and Foreign Key constraints (ON DELETE RESTRICT).				
Acceptance Criteria: System blocks deletion and displays an alert: "Cannot delete this book because it is linked to existing loan history records."				
Special Requirements: Requires a book that has at least one entry in the `loans` table.				
Test Result: Pass				
Issues/Comments: N/A				
Tested By: Farhan		**Date Tested:** March 11, 2026		
Test Case Details				
Screen/Field/Form	**Action**	**Data Value**	**Expected Result**	**Actual Result**
index.php?page=books	Click "Delete" on a book that is currently on loan.	**Book ID:** 1	Redirect to inventory with a red error alert at the top.	Worked as expected.

Test Case #: 9				
Test Description: Verify that a staff member can register a new client into the system.				
Use Case: Add Client				
Functional Requirements Tested: Client Management – Maintain client profiles (add).				
Acceptance Criteria: The client details are saved and the new client appears in the Client Directory.				
Special Requirements: User must be logged in as Staff or Admin.				
Test Result: Pass				
Issues/Comments: N/A				
Tested By: Farhan		**Date Tested:** March 11, 2026		
Test Case Details				
Screen/Field/Form	**Action**	**Data Value**	**Expected Result**	**Actual Result**
index.php?page=clients_create	Fill in Name, Email,	**Name:** Jane Doe **Email:** jane@example.com	Redirect to index.php?page=clients	Worked as expected.

	and Phone, then click "Save Client".	**Phone:** 902-555-0123	with the new client listed.	

Test Case #: 10				
Test Description: Test Description: Verify the accuracy of the "Overdue Books Report" by identifying books where the return date is NULL and the due date has passed.				
Use Case: Generate Overdue Books Report				
Functional Requirements Tested: Reporting – Identification of overdue inventory.				
Acceptance Criteria: The report accurately displays the book title, client name, and the specific past-due date.				
Special Requirements: Requires a loan record in the database where due_date < CURDATE() and return_date is NULL.				
Test Result: Pass				
Issues/Comments: N/A				
Tested By:Farhan		**Date Tested:** March 11, 2026		
Test Case Details				
Screen/Field/Form	**Action**	**Data Value**	**Expected Result**	**Actual Result**
index.php?page=overdue_report	Navigate to the Overdue Report dashboard.	**Current Date:** 2026-03-11 **Due Date:** 2026-03-01	Book "Clean Code" appears in the list assigned to "Jane Doe".	Worked as expected.

Test Case #: 11		
Test Description: Verify that a staff member can successfully checkout a book to a client.		
Use Case: Checkout Book		
Functional Requirements Tested: Loan Tracking – Record book check-outs including client, dates, and status.		
Acceptance Criteria: A new record is added to the `loans` table and appears as "Active" in the Loan Registry.		
Special Requirements: Requires at least one available book and one registered client.		
Test Result: Pass		
Issues/Comments: N/A		
Tested By: Farhan	**Date Tested:** March 11, 2026	
Test Case Details		

Screen/Field/Form	Action	Data Value	Expected Result	Actual Result
index.php?page=loan_checkout	Select a book and a client from the dropdown menus and set a due date.	**Book:** The Pragmatic Programmer **Client:** Jane Doe **Due Date:** 2026-03-25	Redirect to index.php?page=loans; new active loan is visible.	Worked as expected.

Test Case #: 12	
Test Description: Attempt to checkout a book that is already currently on loan (Active status) to another client.	
Use Case: Checkout Book	
Functional Requirements Tested: Testing system logic for book availability verification before loan creation.	
Acceptance Criteria: System should block the transaction and display an error: "Book is not available or there was an error."	
Special Requirements: Requires a book record that is currently checked out (no return date in `loans` table).	
Test Result: Fail	
Issues/Comments: The application logic failed to prevent the creation of a duplicate active loan for the same book ID.	
Tested By: Farhan	**Date Tested:** March 11, 2026

Test Case Details

Screen/Field/Form	Action	Data Value	Expected Result	Actual Result
index.php?page=loan_checkout	Select a book that is already on loan and click "Confirm Checkout".	**Book:** The Pragmatic Programmer (ID: 2) **Client:** Peter MacDonald	Receive error: "Book is not available or there was an error."	No error message; a new active loan was successfully added for the same book.

Test Case #: 13

Test Description: Test Description: Verify that a staff member can successfully process the return of a book, updating the loan status and recording the return date.				
Use Case: Return Book				
Functional Requirements Tested: Loan Tracking – Record book returns and update status.				
Acceptance Criteria: The loan record is updated with the current date in the `return_date` field, and the status in the Loan Registry changes from "Active" to "Returned".				
Special Requirements: Requires at least one "Active" loan record to exist in the database.				
Test Result: Pass				
Issues/Comments: N/A				
Tested By: Farhan			**Date Tested:** March 11, 2026	
Test Case Details				
Screen/Field/Form	**Action**	**Data Value**	**Expected Result**	**Actual Result**
index.php?page=loan_return&id=1	Click "Mark Returned" on an active loan and confirm on the secondary processing screen.	**Loan ID: 1**	Redirect to Loan Registry; status shows "Returned on [Current Date]".	Worked as expected.

Test Case #: 14				
Test Description: Test Description: Verify that the system automatically logs out a user after a period of inactivity to maintain system security.				
Use Case: Security / Session Management				
Functional Requirements Tested: Security – Inactivity timeout logic.				
Acceptance Criteria: After 600 seconds of no activity, the next page request should redirect the user to the login screen with a timeout message.				
Special Requirements: User must be logged in.				
Test Result: Pass				
Issues/Comments: Timeout duration is currently set to 60 seconds for testing/demonstration purposes.				
Tested By: Farhan			**Date Tested:** March 11, 2026	
Test Case Details				
Screen/Field/Form	**Action**	**Data Value**	**Expected Result**	**Actual Result**
Any protected page (e.g., index.php?page=books)	Log in, wait for 65 seconds without	**Timeout Duration: 600s**	Redirect to login.php?timeout=1; yellow alert warns	Worked as expected.

	clicking any links, then refresh the page.		about inactivity logout.	